

Regression — Gradient Descent

Use this when the exam says: "use *gradient descent*".

How it works: Iteratively nudge weights in the direction that reduces cost.

$$W := W - (1r / m) * \Phi^T(\Phi W - t)$$

What you need to set:

- `learning_rate` — how big each step is (try `0.01`, `0.1`, `1`)
- `epochs` — how many steps to take (try `1000` – `2000`)

Always:

- Normalize features before gradient descent — otherwise features on different scales cause slow/broken convergence
- Plot the cost history to confirm it went down and flattened (converged)

Case	Design matrix Φ row
Linear (1 var)	<code>[1, x]</code>
Polynomial degree d	<code>[1, x, x², ..., x^d]</code>
Multivariate (n vars)	<code>[1, x1, x2, ..., xn]</code>

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
```

Shared functions — copy these into the exam

```
In [2]: def computeCost(X, Y, W):
    m = Y.size
    error = X @ W - Y          # predictions minus actual
    C = (1/(2*m)) * np.sum(error**2)
    return C

def gradientDescent(X, Y, W, learning_rate, epochs):
    m = Y.size
    C_history = []
    for i in range(epochs):
        error = X @ W - Y
        grad = (1/m) * (X.T @ error)
        W = W - learning_rate * grad
```

```
C_history.append(computeCost(X, Y, W))
return W, np.array(C_history)
```

Case 1 — Linear Regression (1 variable)

```
In [ ]: # --- dummy data ---
np.random.seed(0)
x_train = np.linspace(0, 10, 50)
t_train = 3*x_train + 2 + np.random.randn(50)*3
m = x_train.shape[0]
# -----

# Normalize the feature
mu_x = x_train.mean()
sigma_x = x_train.std()
x_norm = (x_train - mu_x) / sigma_x

# Design matrix: [1, x_norm]
Phi = np.column_stack([np.ones(m), x_norm]) # shape (m, 2)
Y = t_train.reshape(-1, 1) # column vector (m, 1)

print('Phi shape:', Phi.shape)
print('Y shape: ', Y.shape)
```

```
In [ ]: learning_rate = 0.01
epochs = 1500
W_init = np.zeros((2, 1)) # start at zero

W, C_history = gradientDescent(Phi, Y, W_init, learning_rate, epochs)
print('W:', W.flatten().round(4)) # [intercept, slope in normalized space]

# Plot cost convergence – must go down and flatten
plt.plot(C_history)
plt.title('Cost through iterations (should decrease and flatten)')
plt.xlabel('Epoch')
plt.ylabel('Cost')
plt.grid(True, alpha=0.3)
plt.show()
```

```
In [ ]: t_pred = Phi @ W
sigma_pred = np.sqrt(np.mean((Y - t_pred)**2)) # RMSE
print(f'RMSE (expected uncertainty): ±{sigma_pred:.4f}')

plt.scatter(x_train, t_train, color='steelblue', edgecolors='navy', s=40, label='Train data')
plt.plot(x_train, t_pred, 'r-', linewidth=2.5, label='Linear fit')
plt.xlabel('x')
plt.ylabel('t')
plt.title(f'Linear Regression (Gradient Descent) – RMSE={sigma_pred:.3f}')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Case 2 — Polynomial Regression (degree d)

```
In [ ]: # --- dummy quadratic data ---
np.random.seed(0)
x_train = np.linspace(-3, 3, 50)
t_train = 2*x_train**2 - x_train + 1 + np.random.randn(50)*2
m = x_train.shape[0]
# -----

DEGREE = 2 # change based on scatter plot shape

# Normalize the feature first
mu_x = x_train.mean()
sigma_x = x_train.std()
x_norm = (x_train - mu_x) / sigma_x

def build_design_matrix(x, degree):
    # Each row = [1, x, x^2, ..., x^degree]
    return np.column_stack([x**i for i in range(degree + 1)])

Phi = build_design_matrix(x_norm, DEGREE) # shape (m, degree+1)
Y = t_train.reshape(-1, 1) # (m, 1)

print('Phi shape:', Phi.shape)
```

```
In [ ]: learning_rate = 0.01
epochs = 2000
W_init = np.zeros((DEGREE + 1, 1))

W, C_history = gradientDescent(Phi, Y, W_init, learning_rate, epochs)
print('W:', W.flatten().round(4))

plt.plot(C_history)
plt.title('Cost through iterations')
plt.xlabel('Epoch')
plt.ylabel('Cost')
plt.grid(True, alpha=0.3)
plt.show()
```

```
In [ ]: t_pred = Phi @ W
sigma_pred = np.sqrt(np.mean((Y - t_pred)**2)) # RMSE
print(f'RMSE (expected uncertainty): ±{sigma_pred:.4f}')

# For the plot: build a smooth line using normalized x values
x_line = np.linspace(x_train.min()-0.1, x_train.max()+0.1, 300)
x_line_norm = (x_line - mu_x) / sigma_x # normalize with TRAINING DATA
t_line = build_design_matrix(x_line_norm, DEGREE) @ W

plt.figure(figsize=(9, 5))
plt.fill_between(x_line, t_line.flatten()-sigma_pred, t_line.flatten()+sigma_pred,
                 alpha=0.2, color='red', label=f'±1σ = ±{sigma_pred:.2f}')
plt.fill_between(x_line, t_line.flatten()-2*sigma_pred, t_line.flatten()+2*sigma_pr
```

```

        alpha=0.1, color='red', label='±2σ (95%)')
plt.scatter(x_train, t_train, color='steelblue', edgecolors='navy',
            s=40, alpha=0.8, zorder=5, label='Training data')
plt.plot(x_line, t_line, 'r-', linewidth=2.5, zorder=4, label=f'Degree-{DEGREE} fit')
plt.xlabel('x1')
plt.ylabel('t')
plt.title(f'Polynomial Regression degree {DEGREE} (Gradient Descent) - RMSE={sigma_}')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

```

Case 3 — Multivariate Linear Regression (n variables)

```

In [ ]: # --- dummy multivariate data (2 features) ---
np.random.seed(0)
X_train = np.column_stack([np.random.randn(47)*500 + 2000,
                           np.random.randint(1, 6, 47)])
t_train = 100*X_train[:, 0] + 5000*X_train[:, 1] + np.random.randn(47)*10000
m, n = X_train.shape
# -----

# Normalize (essential for gradient descent with multi-scale features)
mean = X_train.mean(axis=0) # (n,)
std = X_train.std(axis=0) # (n,)
X_norm = (X_train - mean) / std

# Add bias column
Phi = np.hstack([np.ones((m, 1)), X_norm]) # shape (m, n+1)
Y = t_train.reshape(-1, 1) # (m, 1)

print('Phi shape:', Phi.shape)

```

```

In [ ]: learning_rate = 0.015
epochs = 1200
W_init = np.zeros((n + 1, 1)) # (3, 1) for 2 features + bias

W, C_history = gradientDescent(Phi, Y, W_init, learning_rate, epochs)
print('W:', W.flatten().round(2))

plt.plot(C_history)
plt.title('Cost through iterations')
plt.xlabel('Epoch')
plt.ylabel('Cost')
plt.grid(True, alpha=0.3)
plt.show()

```

```

In [ ]: # Predict a new sample - normalize with TRAINING mean and std
x_new = np.array([1650, 3])
x_new_norm = (x_new - mean) / std # TRAINING stats
x_new_bias = np.append(1, x_new_norm) # add bias

```

```
t_new_pred = x_new_bias @ W
print(f'Prediction for {x_new}: {float(t_new_pred):.2f}')
```

Summary

```
def computeCost(X, Y, W):
    m = Y.size
    error = X @ W - Y
    C = (1/(2*m)) * np.sum(error**2)
    return C

def gradientDescent(X, Y, W, learning_rate, epochs):
    m = Y.size
    C_history = []
    for i in range(epochs):
        error = X @ W - Y
        grad = (1/m) * (X.T @ error)
        W = W - learning_rate * grad
        C_history.append(computeCost(X, Y, W))
    return W, np.array(C_history)
```

Checklist:

- Normalize features before building Phi
- `Y = t.reshape(-1, 1)` — gradient descent expects column vector
- `W_init = np.zeros((num_cols_in_Phi, 1))`
- Plot cost history — must decrease and flatten
- Use TRAINING mean / std when predicting on test data

If cost diverges (goes up): learning rate is too high — try 10× smaller.

If cost barely decreases: learning rate is too small — try 10× larger.